Algorithm: Step 1 – Understand the User Prompt

Step 1: Prompt Parsing and Structured Representation

1.1 Ingest raw natural language prompt $\mathcal{T}_{\text{user}}$ via client interface and extract core objective.

1.2 Retrieve persistent session context and workspace history from browser IndexedDB (`PrismDB`):

- **1.2.1** Load prior interaction turns, agent memory checkpoints, and active user configurations.
- **1.2.2** Resolve local file system references, target project repositories, and access tokens.

1.3 Construct strongly-typed operational execution tuple:

$$T = \langle \text{Goal, Constraints, Context, } \mathcal{M}_{\text{tools}}, \Phi_{\text{out}} \rangle$$

- **1.3.1** Parse explicit operational constraints (deadlines, schema specifications, output formats).
- **1.3.2** Bind authorized Model Context Protocol (MCP) tool capabilities and environment permissions.

1.4 Perform input boundary and security screening:

- **1.4.1** Evaluate prompt against token limits and scan for prompt injection patterns.
- **1.4.2** Flag state-mutating actions requiring mandatory human-in-the-loop authorization.

Algorithm: Step 2 – Classify the Request

Step 2: Intent Classification and Capability Check

2.1 Categorize user prompt into functional operational domains:

$$\text{Domain} \in \{\text{Productivity/Automation, Document Synthesis, Web Research, Code Engineering}\}$$

- **2.1.1** Compute dense semantic embeddings and syntactic keyword vectors for input prompt.
- **2.1.2** Route through multi-class classifier head $\hat{\mathbf{y}}_{\text{intent}}$ to determine target capability domain.

2.2 Profile required system capabilities and operational risk levels:

- **2.2.1** Determine required subsystem resources (live web search, local filesystem I/O, terminal execution).
- **2.2.2** Enforce safety categorization: Read-Only (autonomous execution) vs. State-Mutating (approval-gated).

2.3 Select optimal execution pathway:

- **2.3.1** Route single-step queries to direct, low-latency single-model inference pipeline.
- **2.3.2** Dispatch multi-step complex requests to DAG Planner agent for task decomposition.

Step 3: DAG Decomposition and Model Routing

3.1 Deconstruct objective into Directed Acyclic Graph $\mathcal{G} = (V, E)$:

- **3.1.1** Instantiate discrete task vertices $v_i \in V$ with atomic execution contracts and output targets.
- **3.1.2** **Data Dependencies as Directed Edges** ($e = \langle v_i, v_j \rangle \in E \iff \mathcal{O}_i \cap \mathcal{I}_j \neq \emptyset$): Task v_j requires input artifacts produced by v_i (e.g., synthesis requires scraped research).
- **3.1.3** **Execution Barrier:** Consumer v_j remains blocked until producer v_i delivers $\mathcal{O}_i$, guaranteeing tasks never run on missing or partial data.

3.2 Compute topological task execution schedule $\pi = \mathrm{TopologicalSort}(\mathcal{G})$:

- **3.2.1** Identify independent vertices (in-degree $= 0$) for concurrent parallel dispatch ($K \leq 3$).
- **3.2.2** Schedule downstream vertices dynamically upon upstream output event emission.

3.3 Solve multi-objective provider routing optimization for each sub-task v_i:

$$\text{Provider}^* = \arg\min_m \left(\lambda_{\text{cost}} \, \text{Cost}_m + \lambda_{\text{latency}} \, \text{Latency}_m \right) \quad \text{s.t. } \text{Quality}_m \geq \tau$$

- **3.3.1** Route high-throughput, latency-critical nodes to LPU backends (Groq / Llama-3).
- **3.3.2** Route deep reasoning and code generation tasks to frontier backends (OpenAI GPT-4o, Claude 3.5).

Algorithm: Step 4 – Task Assignment and Agent Selection

Step 4: Role-Based Task Assignment and Dispatching

4.1 Query specialist agent registry and capability profiles:

$$\mathcal{A} = \{\text{Planner, Assistant, Researcher, Coder, Validator}\}$$

- **4.1.1** Evaluate persona system prompts, specialized instruction boundaries, and assigned MCP tools.
- **4.1.2** Verify agent worker availability and monitor concurrent thread utilization.

4.2 Execute capability matching algorithm for each sub-task node v_i:

$$\text{Agent}^* = \arg\max_{a \in \mathcal{A}} \ \text{CosineSimilarity}\big(\textbf{Req}(v_i),\ \textbf{Cap}(a)\big)$$

- **4.2.1** Match information gathering requirements (Tag $\ni$ `web_search`) to Researcher agent.
- **4.2.2** Match software synthesis and execution (Tag $\ni$ `python, cli`) to Coder agent.
- **4.2.3** Match quality assurance and policy compliance (Tag $\ni$ `audit, verify`) to Validator agent.

4.3 Manage task lifecycle states in dispatcher queue:

- **4.3.1** Place sub-tasks in `PENDING` state until all parent dependency inputs resolve.
- **4.3.2** Transition verified nodes to `READY` and dispatch execution payloads to assigned agents.

Step 5: Agent Decision Cycle and Inter-Agent Communication

5.1 Execute autonomous Reasoning-Action (ReAct) cycle for each dispatched agent:

$$\text{State } s_t \longrightarrow \textbf{Observe} \longrightarrow \textbf{Reason} \longrightarrow \textbf{Act} \longrightarrow \textbf{Reflect}$$

- **5.1.1** **Observe:** Ingest assigned sub-task description, operational constraints, and parent inputs.
- **5.1.2** **Reason:** Formulate intermediate chain-of-thought rationale to plan next tool invocation.
- **5.1.3** **Act:** Dispatch structured JSON-RPC tool calls to external MCP servers or formulate text.
- **5.1.4** **Reflect:** Inspect execution exit codes and output schemas; initiate local correction on errors.

5.2 Transmit intermediate artifacts via DAG edge message passing:

- **5.2.1** Route validated producer artifact $\mathcal{O}_i$ directly into consumer input buffer $\mathcal{I}_j$.
- **5.2.2** Enforce strict artifact isolation to prevent cross-task context contamination and token inflation.

5.3 Synchronize shared state via local blackboard memory:

- **5.3.1** Commit global project state, conversation turns, and file handles to IndexedDB (`PrismDB`).
- **5.3.2** Dispatch event-driven signals upon node resolution to awaken dependent downstream workers.

Step 6: Specialist Execution Workflows – Planner & Assistant

6.1 Execute Planner Agent workflow (System Architect & Coordinator):

- **6.1.1** Ingest user objective string and conversation context from `PrismDB`.
- **6.1.2** Decompose objective into atomic independent and dependent sub-goals.
- **6.1.3** Construct DAG graph vertices V, dependency edges E, and capability tags.
- **6.1.4** Solve provider routing optimization and enqueue topological schedule into Dispatcher.

6.2 Execute Assistant Agent workflow (Productivity & Communication):

- **6.2.1** Retrieve user communication preferences, profiles, and reference documents from storage.
- **6.2.2** Formulate structured response drafts, templates, or executive summaries.
- **6.2.3** Interface with Google Workspace MCP tools (Gmail draft creation, Drive document retrieval).
- **6.2.4** Emit verified communication draft; enforce approval gate prior to external message transmission.

Step 7: Specialist Execution Workflows – Researcher, Coder & Validator

7.1 Execute Researcher Agent workflow (Information Retrieval & Synthesis):
- **7.1.1** Formulate optimized web search queries and invoke Browser MCP server endpoints.
- **7.1.2** Scrape target web documents, parse DOM trees, and extract relevant contextual passages.
- **7.1.3** Deduplicate findings and compile structured Markdown report with verified source citations.

7.2 Execute Coder Agent workflow (Software Synthesis & Execution):
- **7.2.1** Inspect local repository tree via Filesystem MCP and synthesize target source code files.
- **7.2.2** Execute scripts inside Sandboxed Terminal CLI and inspect exit codes and error streams.
- **7.2.3** Analyze stack traces and compiler lints to autonomously self-correct syntax and runtime bugs.

7.3 Execute Validator Agent workflow (Quality Auditing & Safety Gate):
- **7.3.1** Scan candidate outputs against safety policies, token invariants, and functional constraints.
- **7.3.2** Emit binary verification signal: **PASS** to release artifact, or **FAIL** with error diagnostics.

Step 8: Standardized Tool Execution via Model Context Protocol

8.1 Interface agents to external tools via Model Context Protocol (MCP):

- **8.1.1** Standardize client-server communication using JSON-RPC protocol specifications.
- **8.1.2** Enable dynamic tool discovery: inspect exposed tool schemas, input parameters, and return types.
- **8.1.3** Decouple agent reasoning logic from external tool implementation details.

8.2 Orchestrate multi-domain tool execution ecosystem:

- **8.2.1** **System & Dev Tools:** Local Filesystem (file I/O), Sandboxed CLI (isolated shell), GitHub API.
- **8.2.2** **Productivity Tools:** Google Workspace MCP (Gmail messaging, Google Drive document access).
- **8.2.3** **Web Intelligence:** Browser MCP for real-time document search, retrieval, and scraping.

8.3 Enforce Human-in-the-Loop (HITL) authorization policy:

- **8.3.1** Permit autonomous execution for non-destructive, read-only tool queries.
- **8.3.2** Intercept state-mutating actions (file modification, email dispatch, CLI execution) for user approval.

Step 9: Context Compaction and Shared Memory Management

9.1 Address context window constraints and token consumption limits:

- **9.1.1** Mitigate fixed LLM context window boundaries to eliminate token overflow and latency degradation.
- **9.1.2** Prevent quadratic attention computation overhead across long multi-agent interaction sessions.

9.2 Execute dynamic context compaction and artifact pruning:

$$C_{t+1} = \mathrm{Prune}\big(C_t \cup \Delta\text{Artifacts} \setminus \text{Stale Tokens}\big)$$

- **9.2.1** Forward only verified parent output artifacts $\mathcal{O}_i$ to dependent downstream child nodes.
- **9.2.2** Evict intermediate reasoning scratchpads, redundant tool telemetry, and discarded draft tokens.

9.3 Persist global execution state using client-side database (`PrismDB`):

- **9.3.1** Commit session turns, task graphs, and tool execution logs to browser IndexedDB via Dexie.js.
- **9.3.2** Ensure client data privacy, full offline execution persistence, and instant crash recovery.

Algorithm: Step 10 – ORPO Preference Model & Verification

Step 10: Single-Stage Preference Alignment (ORPO) and Verification

10.1 Single-stage reference-free preference alignment formulation (ORPO):

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \lambda_{\text{OR}} \cdot \mathcal{L}_{\text{OR}} \quad \text{where} \quad \mathcal{L}_{\text{OR}} = -\log \sigma \left(\log \left[\frac{\text{odds}_\theta(y_w \mid x)}{\text{odds}_\theta(y_l \mid x)} \right] \right)$$

- **10.1.1** Jointly maximize chosen likelihood while penalizing rejected tokens (2× memory savings).
- **10.1.2** Supervise via UltraFeedback pairs $\mathcal{D} = \{(x, y_w, y_l)\}$, bypassing unstable RLHF loops.

10.2 Enforce runtime quality gating via preference score barrier $\mathcal{S}_{\text{ORPO}}(x, y) \geq \tau_{\text{approval}}$:

- **10.2.1** Audit deliverables using fine-tuned model (`reward_orpo_v2`, Qwen2.5-1.5B LoRA).
- **10.2.2** Score candidate outputs on instruction adherence, structural constraints, and safety guardrails.

10.3 Handle quality failures via targeted DAG re-planning:

- **10.3.1** Capture diagnostic error traces and schema deviations when output falls below τ_{approval}.
- **10.3.2** Dispatch error logs to DAG Planner for targeted branch re-planning without system abort.

Algorithm: Step 11 – Final Deliverable Synthesis and Commit

Step 11: Artifact Assembly and Global State Commit

11.1 Aggregate and assemble multi-agent deliverables:

11.1.1 Harvest verified partial output artifacts $\mathcal{O}_i$ from all terminal nodes in the completed DAG.

11.1.2 Coalesce research summaries, verified code diffs, and formatted text into unified final deliverable.

11.2 Calculate end-to-end composite delivery quality metric:

$$\text{Delivery Quality } D = \text{Correctness} \times \text{Completeness} \times \text{Timeliness}$$

11.2.1 Apply multiplicative scoring penalty: failure in any single dimension sharply penalizes overall quality.

11.2.2 Log quality vector to persistent performance ledger to calibrate future provider routing models.

11.3 Commit finalized session state with ACID guarantees:

11.3.1 Atomically write completed conversation records, DAG execution graphs, and tool receipts to `PrismDB`.

11.3.2 Record end-to-end telemetry: aggregate latency, per-agent token consumption, and audit receipts.

Step 12: Output Delivery and Reactive Workspace Presentation

12.1 Synthesize and package final deliverable artifacts:

- **12.1.1** Compile structured digital artifacts (Markdown reports, verified code patches, email/document drafts).
- **12.1.2** Generate comprehensive audit receipt containing agent provenance, execution duration, and tool logs.

12.2 Render presentation layer via multi-pane workspace interface:

- **12.2.1** Update interactive DAG HUD with real-time node states (`Queued` → `Running` → `Success` / `Failed`).
- **12.2.2** Mount Artifact Viewer canvas providing rich Markdown rendering, syntax highlighting, and export controls.
- **12.2.3** Present interactive approval cards for pending external actions with explicit authorization controls.

12.3 Synchronize client state reactively via local database transactions:

- **12.3.1** Commit verified state transitions and output artifacts directly into browser IndexedDB (`PrismDB`).
- **12.3.2** Trigger Dexie `useLiveQuery()` observers to update UI components reactively upon local database commits.